

[\(https://learnopencv.com/\)](https://learnopencv.com/)[Home \(https://learnopencv.com/\)](#) > [Image Segmentation \(https://learnopencv.com/category/image-segmentation/\)](https://learnopencv.com/category/image-segmentation/) > [Semantic Segmentation using KerasCV DeepLabv3+](#)

100%

Kunal Dawn (<https://learnopencv.com/author/kunal/>) on October 3, 2023

Semantic Segmentation using KerasCV DeepLabv3+

Fine-tuning DeepLabv3+ from KerasCV for Semantic Segmentation

[Image Segmentation \(https://learnopencv.com/category/image-segmentation/\)](https://learnopencv.com/category/image-segmentation/)[Keras \(https://learnopencv.com/category/keras/\)](https://learnopencv.com/category/keras/)[Segmentation \(https://learnopencv.com/category/segmentation/\)](https://learnopencv.com/category/segmentation/)[Tensorflow \(https://learnopencv.com/category/tensorflow/\)](https://learnopencv.com/category/tensorflow/)

100%



(<https://learnopencv.com/wp-content/uploads/2023/10/KerasCV-DeepLabV3-blog-post-feature.gif>)

The KerasCV series continues with this second article. Continuing from the previous post, where we discussed [Object Detection using KerasCV YOLOv8](https://learnopencv.com/object-detection-using-kerascv-yolov8/) (<https://learnopencv.com/object-detection-using-kerascv-yolov8/>), this article discusses solving a semantic segmentation problem by fine-tuning the **KerasCV DeepLabv3+** model.

DeepLabv3+ is a prevalent semantic segmentation model that finds use across various applications in image segmentation, such as medical imaging, autonomous driving, etc.

KerasCV, too, has integrated DeepLabv3+ into its library. In this blog post, we shall extensively discuss how to leverage DeepLabv3+ and fine-tune it on our custom data. Specifically, we will use the following ImageNet pre-trained backbones as feature extractors for fine-tuning DeepLabv3+:

- **ResNet50_V2**
- **EfficientNetv2_small**

Finally, we will also compare the results across these models.



Table of Contents

1. The Satellite Water Bodies Semantic Segmentation Dataset

- 1.1. Pixel Differences and Mislabeled Annotations
- 1.2. Presence of White Pixels at Boundaries of Ground Truth Masks
- 1.3. Irrelevant Image Samples and Masks

2. Dataset Preparation for Semantic Segmentation through KerasCV

- 2.1. Download Image Data and Ground Truth Annotations
- 2.2. Hyper-parameter Configurations for KerasCV DeepLabv3+
- 2.3. Dataset Preparation for Semantic Segmentation

3. Ground Truth Visualizations for Semantic Segmentation through KerasCV

4. Data Augmentation for KerasCV DeepLabv3+ Training Pipeline

5. The KerasCV DeepLabv3+ Model

- 5.1. Model Callbacks for KerasCV DeepLabv3+

6. Evaluation Metrics

7. Training KerasCV DeepLabv3+ on the Satellite Images of Water Bodies Dataset

8. Predictions Visualization from KerasCV DeepLabv3+

9. Summary and Conclusion

10. References

100%





100%

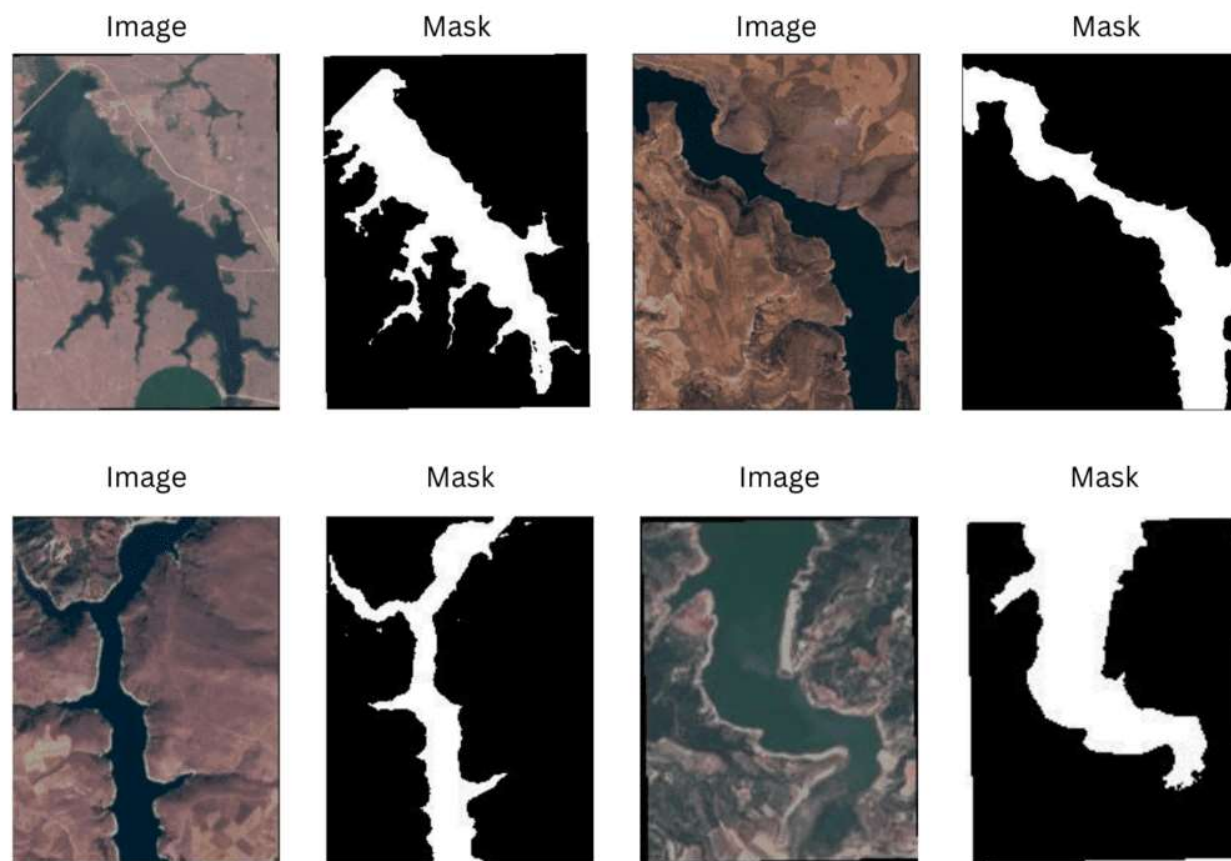


Watch on

The Satellite Water Bodies Semantic Segmentation Dataset

We will fine-tune our DeepLabv3 model on satellite images of waterbodies. To give a brief context on the dataset, it consists of images captured by the Sentinel-2 Satellite. Each image comes with a binary mask where white represents water and black represents the background. The masks were generated by calculating the NWDI (Normalized Water Difference Index), which is frequently used to detect and measure vegetation in satellite images, but a greater threshold was used to detect water bodies.

Let's take a look at a few samples.



(<https://learnopencv.com/wp-content/uploads/2023/10/ground-truth-samples-for-kerascv-deeplabv3.png>)

Ground Truth Samples

There are a total of **2841** samples. We will choose a **95-5** split for the train and validation splits. However, before we proceed with the data preparation, there are a few issues with the data worth mentioning.

Pixel Differences and Mislabeled Annotations

Although we have mentioned that the dataset contains binary masks as the ground truth labels, they are not **genuinely binary** because there are pixels that are not entirely **0** or **255**. There are quite a few gray pixels in the masks. The issue can be resolved through [image](#)

[thresholding](https://learnopencv.com/opencv-threshold-python-cpp/) (<https://learnopencv.com/opencv-threshold-python-cpp/>). Specifically, we will threshold all pixel values greater than **200** to **255** and **0** below the threshold, which essentially leaves us with precisely two classes for the labels:

- Class 0: Background
- Class 1: Water bodies (pixel values being 255).

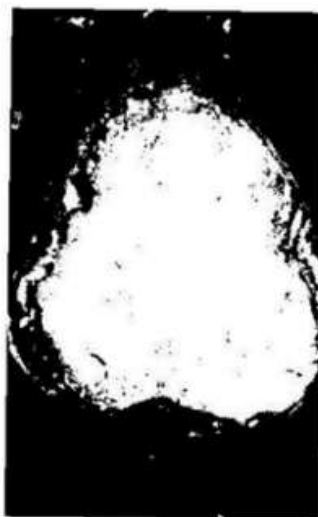
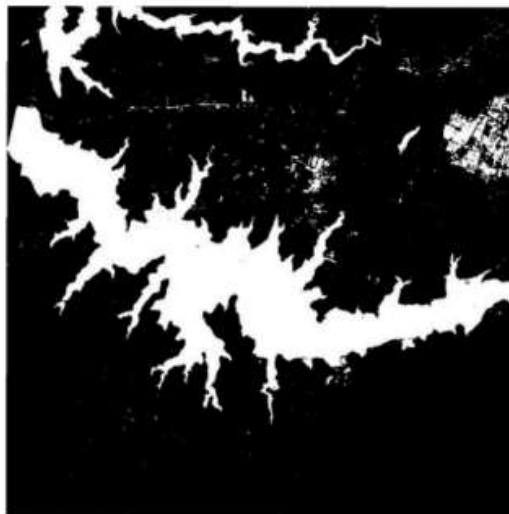
We can also perform further morphological transformations to reduce the noise, but we shall stick to thresholding for now.

Even after thresholding, there are some mislabelled annotations, as shown below.

100%



100%



(<https://learnopencv.com/wp-content/uploads/2023/10/mislabeled-annotations-in-dataset.png>)

Mislabeled annotations in Ground Truth Masks

100%



In the first two images and their corresponding masks, you can see that the buildings and some land cover are labeled as water bodies, which is incorrect. Similarly, in the second image, most of the patch of land is labeled as waterbodies. These anomalies significantly hamper model learning; however, we will keep these samples and observe how our model performs.

Presence of White Pixels at Boundaries of Ground Truth Masks

The samples below depict the boundaries of the masks labeled as waterbodies (having pixel values of **255**), as shown below.



(<https://learnopencv.com/wp-content/uploads/2023/10/white-pixels-at-mask-boundaries.png>)

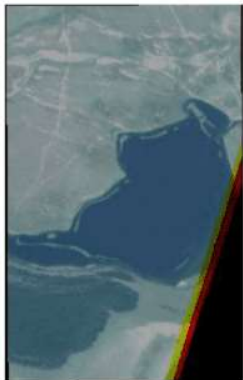
Presence of white pixels in ground truth masks

Both of the masks consist of white pixels at the edges, inadvertently labeled as water bodies, affecting model learning.

Irrelevant Image Samples and Masks

So far, we have discussed anomalies in the dataset where there were partial mislabeled annotations. However, a few samples in the dataset are completely misannotated. The image samples and their corresponding masks are shown below.

Image



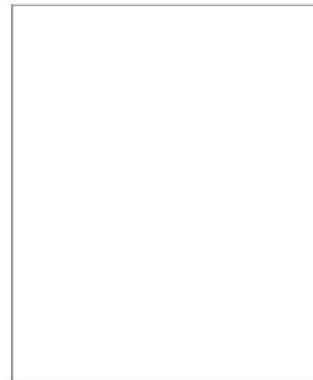
Mask



Image



Mask



Image



Mask



(<https://learnopencv.com/wp-content/uploads/2023/10/complete-missannotations-in-dataset.png>)

Complete mislabeled annotation in Label Masks

As you can observe, the entire ground truth masks across all these samples are white, indicating the presence of water bodies. Ideally, such samples should be omitted during data preparation.

From these discussions, it is evident that real-world datasets are frequently not clean. This could significantly impact model learning during training. Hence, cleaning the data before it is passed to the neural network model is imperative.

Download Code To easily follow along this tutorial, please download code by clicking on the button below. It's FREE!



100%

Dataset Preparation for Semantic Segmentation through KerasCV



Before we start with the data preparation, we need to have **keras_cv** installed first. KerasCV contains modular computer vision components that work natively with TensorFlow, JAX, and PyTorch. It allows seamless customization of models and other training pipelines across major computer vision domains, such as classification, object detection, semantic segmentation, etc. Use the following command to install **keras_cv** .

```
1 | !pip install -q git+https://github.com/keras-team/keras-cv.git --upgrade
```

Once installed, we shall import the following dependencies for the training pipeline.

```
1 | import os
2 | import requests
3 | from zipfile import ZipFile
4 | import glob
5 | from dataclasses import dataclass
6 |
7 | import random
8 | import numpy as np
9 | import cv2
10 |
11 | import tensorflow as tf
12 | import keras_cv
13 |
14 | import matplotlib.pyplot as plt
```

The next step involves maintaining the appropriate seeds for reproducibility.

```
1  def system_config(SEED_VALUE):
2      # Set python `random` seed.
3      # Set `numpy` seed
4      # Set `tensorflow` seed.
5      random.seed(SEED_VALUE)
6      tf.keras.utils.set_random_seed(SEED_VALUE)
7      os.environ["CUDA_VISIBLE_DEVICES"] = '0'
8      os.environ['TF_CUDNN_DETERMINISTIC'] = '1'
9      os.environ['TF_USE_CUDNN'] = "true"
10
11  system_config(SEED_VALUE=42)
```

Download Image Data and Ground Truth Annotations

Begin by downloading the images and the ground truth masks. The `download_and_unzip` utility downloads and extracts the required data.

```
1  def download_and_unzip(url, save_path):
2
3      print("Downloading and extracting assets...", end="")
4      file = requests.get(url)
5      open(save_path, "wb").write(file.content)
6
7      try:
8          # Extract tarfile.
9          if save_path.endswith(".zip"):
10             with ZipFile(save_path) as zip:
11                 zip.extractall(os.path.split(save_path)[0])
12
13             print("Done")
14         except:
15             print("Invalid file")
```

In order to download this, the dataset URL has to be specified. The required data will be extracted once the download has been completed.

```
1 DATASET_URL = r"https://www.dropbox.com/scl/fi/9k8t9619b4x0hegued5c5/Water-Bodies"
2 DATASET_DIR = "Water-Bodies-Dataset"
3 DATASET_ZIP_PATH = os.path.join(os.getcwd(), f"{DATASET_DIR}.zip")
4
5 # Download if dataset does not exists.
6 if not os.path.exists(DATASET_DIR):
7     download_and_unzip(DATASET_URL, DATASET_ZIP_PATH)
8     os.remove(DATASET_ZIP_PATH)
```

Once extracted, the directory will have the following structure.

```
1 Water-Bodies-Dataset/
2 |— Images
3 |   |— water_body_1.jpg
4 |   |— water_body_2.jpg
5 |   |— ... (2841 files)
6 |— Masks
7 |   |— water_body_1.jpg
8 |   |— water_body_2.jpg
9 |   |— ... (2841 files)
```

PyTorch Bootcamp



Getting Started with PyTorch

Join Free PyTorch Bootcamp

👤 10K+ Learners

🕒 8 Hours of Learning

OpenCV Bootcamp



Learn Computer Vision & AI Using OpenCV

Join Free OpenCV Bootcamp

👤 100K+ Learners

🕒 3 Hours of Learning



100%

[View all AI Free Courses >](#)https://opencv.org/university/free-courses/?utm_source=lopcv&utm_medium=blog

Hyper-parameter Configurations for KerasCV DeepLabv3+

Next, we will define the various hyper-parameter settings that will be used throughout our training pipeline.

Data Configuration for KerasCV DeepLabv3

The **DatasetConfig** class takes in the various hyperparameters related to the data, such as the image size and batch size to be used while training, the number of classes being used, and the various augmentation factors such as **random brightness**, **random contrast**, etc.

```

1  @dataclass(frozen=True)
2  class DatasetConfig:
3      IMAGE_SIZE:      tuple = (256, 256)
4      BATCH_SIZE:      int = 16
5      NUM_CLASSES:     int = 2
6      BRIGHTNESS_FACTOR: float = 0.2
7      CONTRAST_FACTOR: float = 0.2

```

Training Configuration for KerasCV DeepLabv3

The training hyper-parameters, such as model backbone, the number of epochs, learning rate, etc., are handled by the **TrainingConfig** class.

```
1 @dataclass(frozen=True)
2 class TrainingConfig:
3     MODEL: str = "resnet50_v2_imagenet"
4     EPOCHS: int = 35
5     LEARNING_RATE: float = 1e-4
6     CKPT_DIR: str = os.path.join("checkpoints_" + "_".join(MODEL.split("_"))
7                                     "deeplabv3_plus_" + "_".join(MODEL.split("_"))
8     LOGS_DIR: str = "logs_" + "_".join(MODEL.split("_")[:2])
```

For the EfficientNetv2_s backbone, we can specify “**efficientnetv2_s_imagenet**” as the feature extractor.

Dataset Preparation for Semantic Segmentation

TensorFlow’s **tf.data** API will be used, which enables us to build complex pipelines using simple, reusable code components. It allows us to handle large amounts of data, perform complex transformations, manage data across multiple formats, etc.

We proceed by specifying the image file paths and the ground truth masks. Next, we randomly shuffle these paths to avoid any unnecessary bias during training.

```
1 data_images = glob.glob(os.path.join(DATASET_DIR, "Images", "*.jpg"))
2 data_masks = glob.glob(os.path.join(DATASET_DIR, "Masks", "*.jpg"))
3
4 # Shuffle the data paths before data preparation.
5 zipped_data = list(zip(data_images, data_masks))
6 random.shuffle(zipped_data)
7 data_images, data_masks = zip(*zipped_data)
8 data_images = list(data_images)
9 data_masks = list(data_masks)
```


Creating tensor slices will combine the images and mask file paths to a TensorFlow Dataset. This is achieved using the `tf.data.Dataset.from_tensor_slices` method.

```
1 | org_data = tf.data.Dataset.from_tensor_slices((data_images, data_masks))
```

100%



Train and Validation Splits

Now that the images and masks are coupled into a TensorFlow dataset, we split them into training and validation paths by maintaining a **95–5** split ratio between them.

```
1 | SPLIT_RATIO = 0.05
2 | # Determine the number of validation samples
3 | NUM_VAL = int(len(data_images) * SPLIT_RATIO)
4 |
5 | # Split the dataset into train and validation sets
6 | train_data = org_data.skip(NUM_VAL)
7 | valid_data = org_data.take(NUM_VAL)
```

Data Loading and Mask Preprocessing

We will now load the images and masks from the Dataset created using the file paths. While loading, we will maintain a **(256, 256)** image resolution for resizing the images and masks. We will threshold the pixel values of the masks ranging above **200** to **255**; and update the pixels to **0** for values lying below **200**. The final step will be to maintain the following dictionary, which maps input images with masks.

```
1 | {"images": image, "segmentation_masks": mask}
```

The `read_image_mask` function loads and resizes the inputs from the file paths. Additionally, it also performs mask thresholding.

100%



```
1 def read_image_mask(image_path, mask=False, size = data_config.IMAGE_SIZE):
2     image = tf.io.read_file(image_path)
3
4     if mask:
5         image = tf.io.decode_image(image, channels=1)
6         image.set_shape([None, None, 1])
7         image = tf.image.resize(images=image, size=size, method = "bicubic")
8
9         image_mask = tf.zeros_like(image)
10        cond = image >= 200
11        updates = tf.ones_like(image[cond])
12        image_mask = tf.tensor_scatter_nd_update(image_mask, tf.where(cond), updates)
13        image = tf.cast(image_mask, tf.uint8)
14
15    else:
16        image = tf.io.decode_image(image, channels=3)
17        image.set_shape([None, None, 3])
18        image = tf.image.resize(images=image, size=size, method = "bicubic")
19        image = tf.cast(tf.clip_by_value(image, 0., 255.), tf.float32)
20
21    return image
```

The **load_data** function uses the **read_image_mask** utility function to perform appropriate pre-processing given the input image and mask paths and return the appropriate dictionary mappings.

```
1 def load_data(image_list, mask_list):
2     image = read_image_mask(image_list)
3     mask = read_image_mask(mask_list, mask=True)
4     return {"images":image, "segmentation_masks":mask}
```

We will now create the training and validation datasets.

```
1 train_ds = train_data.map(load_data, num_parallel_calls=tf.data.AUTOTUNE)
2 valid_ds = valid_data.map(load_data, num_parallel_calls=tf.data.AUTOTUNE)
```

100%



The `unpack_inputs` is a utility function that is used to unpack the inputs from the dictionary format to a tuple (images, segmentation_masks). This will be used later for visualizing the images, segmentation masks, and model predictions.

```
1 def unpack_inputs(inputs):
2     images = inputs["images"]
3     segmentation_masks = inputs["segmentation_masks"]
4     return images, segmentation_masks
```

Ground Truth Visualizations for Semantic Segmentation through KerasCV

The color mapping dictionary `id2color` below defines the mapping between class IDs and RGB colors for visualizations.

```
1 id2color = {
2     0: (0, 0, 0), # Background
3     1: (102, 204, 255), # Waterbodies
4 }
```

The utility function `num_to_rgb` will convert a single-channel mask to an RGB representation for visualization purposes. Each class ID in the single-channel mask will be converted to a different color according to the `id2color` dictionary mapping.

100%



```
1 def num_to_rgb(num_arr, color_map=id2color):
2
3     # single_layer = np.squeeze(num_arr)
4     output = np.zeros(num_arr.shape[:2]+(3,))
5
6     for k in color_map.keys():
7         output[num_arr==k] = color_map[k]
8
9     return output.astype(np.uint8)
```

The `image_overlay` utility is used to overlay the RGB segmented mask on top of the corresponding RGB image.

```
1 # Function to overlay a segmentation map on top of an RGB image.
2 def image_overlay(image, segmented_image):
3
4     alpha = 1.0 # Transparency for the original image.
5     beta = 0.7 # Transparency for the segmentation map.
6     gamma = 0.0 # Scalar added to each sum.
7
8     image = image.astype(np.uint8)
9
10    segmented_image = cv2.cvtColor(segmented_image, cv2.COLOR_RGB2BGR)
11
12    image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
13
14    image = cv2.addWeighted(image, alpha, segmented_image, beta, gamma, image)
15    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
16
17    return image
```

The convenience function `display_image_and_mask` will display the original image, the ground truth mask, and the overlaid ground truth mask on the original image.

Besides, it also offers to plot predicted segmentation mask and have it overlayed on the input image. Note that an optional argument allows you to control whether the ground truth mask is displayed as a grayscale or color image.

100%



100%



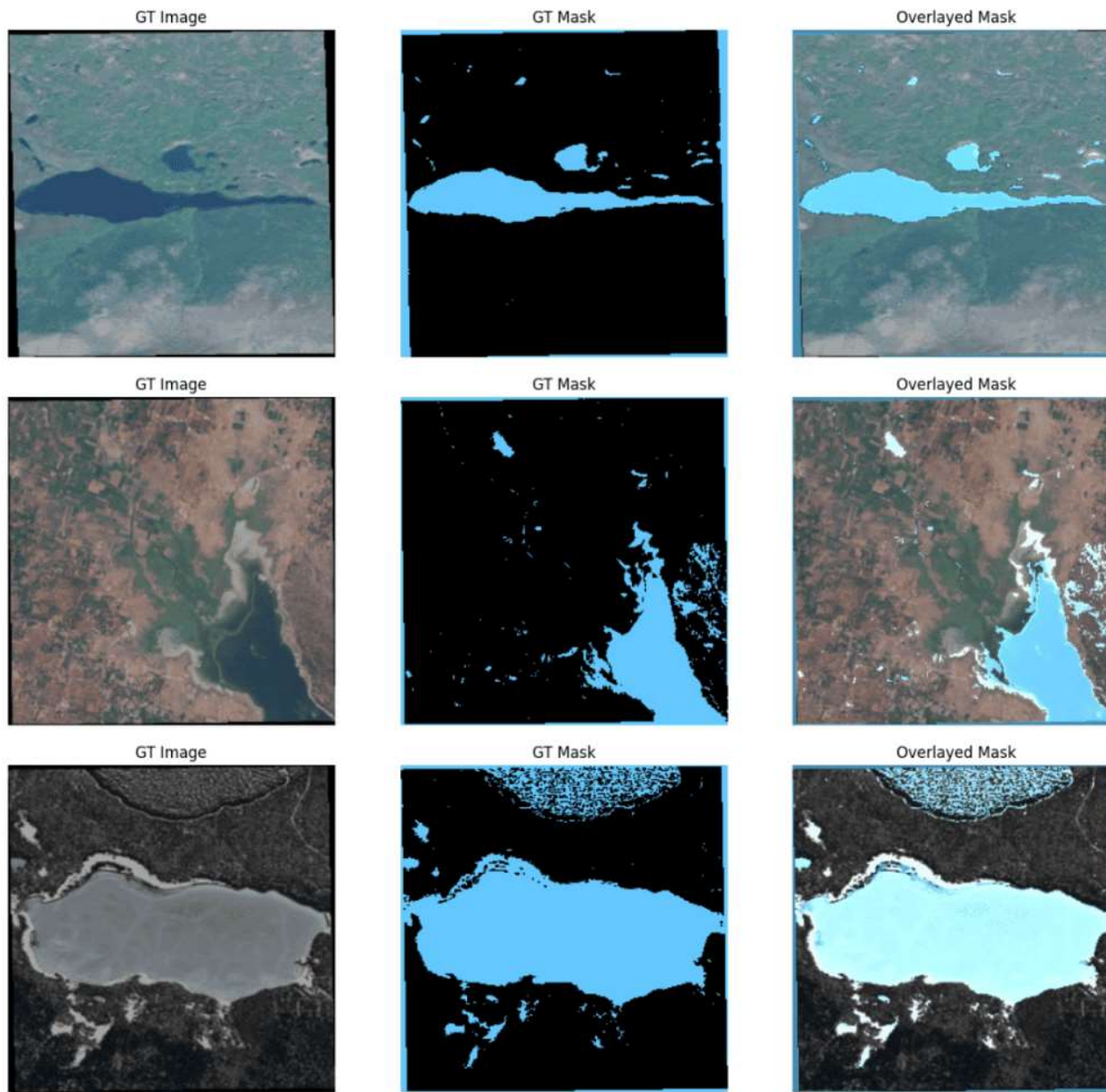
```
1 def display_image_and_mask(data_list, title_list, figsize, color_mask=False, col
2
3     # Create RGB segmentation map from grayscale segmentation map.
4     rgb_gt_mask = num_to_rgb(data_list[1], color_map=color_map)
5     mask_to_overlay = rgb_gt_mask
6
7     if len(data_list)==3:
8         rgb_pred_mask = num_to_rgb(data_list[-1], color_map=color_map)
9         mask_to_overlay = rgb_pred_mask
10
11     # Create the overlaid image.
12     overlaid_image = image_overlay(data_list[0], mask_to_overlay)
13
14     data_list.append(overlaid_image)
15
16     fig, axes = plt.subplots(nrows=1, ncols=len(data_list), figsize=figsize)
17
18     for idx, axis in enumerate(axes.flat):
19         axis.set_title(title_list[idx])
20         if title_list[idx] == "GT Mask":
21             if color_mask:
22                 axis.imshow(rgb_gt_mask)
23             else:
24                 axis.imshow(data_list[1], cmap="gray")
25
26         elif title_list[idx] == "Pred Mask":
27             if color_mask:
28                 axis.imshow(rgb_pred_mask)
29             else:
30                 axis.imshow(data_list[-1], cmap="gray")
31
32         else:
33             axis.imshow(data_list[idx])
34
35         axis.axis('off')
36
37     plt.show()
```

Let us now plot a few ground truth samples.

```
1 plot_train_ds = train_ds.map(unpackage_inputs).batch(3)
2 image_batch, mask_batch = next(iter(plot_train_ds.take(1)))
3
4 titles = ["GT Image", "GT Mask", "Overlaid Mask"]
5
6 for image, gt_mask in zip(image_batch, mask_batch):
7
8     gt_mask = tf.squeeze(gt_mask, axis=-1).numpy()
9     display_image_and_mask([image.numpy().astype(np.uint8), gt_mask],
10                             title_list=titles,
11                             figsize=(16,6),
12                             color_mask=True)
```

100%





(<https://learnopencv.com/wp-content/uploads/2023/10/ground-truth-visualizations-for-kerascv-deeplabv3-finetuning.png>)

Ground Truth Visualization from KerasCV Segmentation Dataset

Data Augmentation for KerasCV DeepLabv3+ Training Pipeline

100%



KerasCV's `RandomFlip`, `RandomBrightness`, and `RandomContrast` will be used to apply image augmentation. Applying augmentations to the validation data should be avoided.

Note: Horizontal Flip is applied by default with a probability factor of **0.5**. Vertical or both horizontal and vertical flips can also be applied by specifying the **mode**. The probability factor is maintained using the **rate** argument.

```
1 augment_fn = tf.keras.Sequential(  
2     [  
3         keras_cv.layers.RandomFlip(),  
4         keras_cv.layers.RandomBrightness(factor=data_config.BRIGHTNESS_FACTOR,  
5                                           value_range=(0, 255)),  
6         keras_cv.layers.RandomContrast(factor=data_config.CONTRAST_FACTOR,  
7                                         value_range=(0, 255)),  
8     ]  
9 )
```

Using the `batch_size` defined in the `TrainingConfig` class, we create the final training and validation datasets. The training set is also shuffled and mapped with the **augment_fn**

```

1 train_dataset = (
2     train_ds.shuffle(data_config.BATCH_SIZE)
3     .map(augment_fn, num_parallel_calls=tf.data.AUTOTUNE)
4     .batch(data_config.BATCH_SIZE)
5     .map(unpackage_inputs)
6     .prefetch(buffer_size=tf.data.AUTOTUNE)
7 )
8
9 valid_dataset = (
10     valid_ds.batch(data_config.BATCH_SIZE)
11     .map(unpackage_inputs)
12     .prefetch(buffer_size=tf.data.AUTOTUNE)
13 )

```

The KerasCV DeepLabv3+ Model

Read this article to understand the [DeepLabv3+ model architecture](https://learnopencv.com/deeplabv3-ultimate-guide/) (<https://learnopencv.com/deeplabv3-ultimate-guide/>) thoroughly.

We will create the DeepLabv3 model using pre-trained ImageNet classification backbones precisely:

- ResNet50_v2
- EfficientNetv2_small

The DeepLab segmentation head will be initialized with random weights.

The following code snippet creates the `deeplabv3_plus` model using the `ResNet50_v2` backbone.

```

1 backbone = keras_cv.models.ResNet50V2Backbone.from_preset(preset = train_config.N
2                                     input_shape=data_config
3                                     load_weights = True)
4 model = keras_cv.models.segmentation.DeepLabV3Plus(
5     num_classes=data_config.NUM_CLASSES, backbone=backbone,
6 )

```

100%



The built model contains approximately **39M** parameters, as vindicated by the model summary.

Model: "deep_lab_v3_plus"

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 256, 256, 3)]	0	[]
model (Functional)	{'P2': (None, 64, 64, 256), 'P5': (None, 8, 8, 2048)}	2355660 8	['input_1[0][0]']
spatial_pyramid_pooling (SpatialPyramidPooling)	(None, 8, 8, 256)	1553817 6	['model[0][1]']
encoder_output_upsampling (UpSampling2D)	(None, 64, 64, 256)	0	['spatial_pyramid_pooling[0][0]']
sequential_1 (Sequential)	(None, 64, 64, 48)	12480	['model[0][0]']
concatenate (Concatenate)	(None, 64, 64, 304)	0	['encoder_output_upsampling[0][0]', 'sequential_1[0][0]']
sequential_2 (Sequential)	(None, 256, 256, 2)	79360	['concatenate[0][0]']

=====
Total params: 39194816 (149.52 MB)
Trainable params: 39145696 (149.33 MB)
Non-trainable params: 49120 (191.88 KB)

(<https://learnopencv.com/wp-content/uploads/2023/10/kerascv-deeplabv3-resnet50v2-summary.png>)

We can load the appropriate weights to build the model using the `EfficientNetv2` backbone.



```

1 backbone = keras_cv.models.EfficientNetV2Backbone.from_preset(preset = train_config.BACKBONE_PRESET,
2                                                                input_shape=data_config.INPUT_SHAPE,
3                                                                load_weights = True)
4 model = keras_cv.models.segmentation.DeepLabV3Plus(
5     num_classes=data_config.NUM_CLASSES, backbone=backbone,
6 )

```

100%

It contains around **30M** parameters, as observed from the summary.

Model: "deep_lab_v3_plus"

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 256, 256, 3)]	0	[]
model (Functional)	{'P2': (None, 64, 64, 48), 'P5': (None, 8, 8, 1280)}	2033136 0	['input_1[0][0]']
spatial_pyramid_pooling (SpatialPyramidPooling)	(None, 8, 8, 256)	9836544	['model[0][1]']
encoder_output_upsampling (UpSampling2D)	(None, 64, 64, 256)	0	['spatial_pyramid_pooling[0][0]']
sequential_1 (Sequential)	(None, 64, 64, 48)	2496	['model[0][0]']
concatenate (Concatenate)	(None, 64, 64, 304)	0	['encoder_output_upsampling[0][0]', 'sequential_1[0][0]']
sequential_2 (Sequential)	(None, 256, 256, 2)	79360	['concatenate[0][0]']

=====
Total params: 30249760 (115.39 MB)

Trainable params: 30092208 (114.79 MB)

Non-trainable params: 157552 (615.44 KB)

(https://learnopencv.com/wp-content/uploads/2023/10/kerascv-deeplabv3-efficientnetv2_s_summary.png)

Model Callbacks for KerasCV DeepLabv3

We will use the **Tensorboard** and **ModelCheckpoint** callbacks to log the losses and metrics and save the model weights. The weights will be saved on the **best validation mIoU** score.



The `get_callbacks` utility defines the callbacks during model training.

```
1  def get_callbacks(  
2      train_config,  
3      monitor="val_mean_iou",  
4      mode="max",  
5      save_weights_only=True,  
6      save_best_only=True,  
7  ):  
8  
9      # Initialize tensorboard callback for logging.  
10     tensorboard_callback = tf.keras.callbacks.TensorBoard(  
11         log_dir=train_config.LOGS_DIR,  
12         histogram_freq=20,  
13         write_graph=False,  
14         update_freq="epoch",  
15     )  
16  
17  
18     # Update file path if saving best model weights.  
19     if save_weights_only:  
20         checkpoint_filepath = train_config.CKPT_DIR  
21  
22     model_checkpoint_callback = tf.keras.callbacks.ModelCheckpoint(  
23         filepath=checkpoint_filepath,  
24         save_weights_only=save_weights_only,  
25         monitor=monitor,  
26         mode=mode,  
27         save_best_only=save_best_only,  
28         verbose=1,  
29     )  
30  
31     return [tensorboard_callback, model_checkpoint_callback]
```

100%



Evaluation Metrics

[Intersection over Union \(IoU\)](https://learnopencv.com/intersection-over-union-iou-in-object-detection-and-segmentation/) (<https://learnopencv.com/intersection-over-union-iou-in-object-detection-and-segmentation/>) is a metric often used in segmentation problems to assess the model's accuracy. It provides a more intuitive basis for accuracy that is not biased by the (unbalanced) percentage of pixels from any particular class. Given two segmentation masks, `A` and `B`, the IoU is defined as follows:

$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

When multiple classes and inferences exist, we assess the model's performance by computing the mean IoU.

The function `mean_iou` below computes the mean IoU that only considers the classes that are present in the ground truth mask or the predicted segmentation map (sometimes referred to as classwise mean IoU). This computation better represents the metric since it only considers the relevant classes. We use this metric computation for mean IoU.

100%



100%



```
1 def mean_iou(y_true, y_pred):
2
3     # Get total number of classes from model output.
4     num_classes = y_pred.shape[-1]
5
6     y_true = tf.squeeze(y_true, axis=-1)
7
8     y_true = tf.one_hot(tf.cast(y_true, tf.int32), num_classes, axis=-1)
9     y_pred = tf.one_hot(tf.math.argmax(y_pred, axis=-1), num_classes, axis=-1)
10
11     # Intersection: |G ∩ P|. Shape: (batch_size, num_classes)
12     intersection = tf.math.reduce_sum(y_true * y_pred, axis=(1, 2))
13
14     # Total Sum: |G| + |P|. Shape: (batch_size, num_classes)
15     total = tf.math.reduce_sum(y_true, axis=(1, 2)) + tf.math.reduce_sum(y_pred, axis=(1, 2))
16
17     union = total - intersection
18
19     is_class_present = tf.cast(tf.math.not_equal(total, 0), dtype=tf.float32)
20     num_classes_present = tf.math.reduce_sum(is_class_present, axis=1)
21
22     iou = tf.math.divide_no_nan(intersection, union)
23     iou = tf.math.reduce_sum(iou, axis=1) / num_classes_present
24
25     # Compute the mean across the batch axis. Shape: Scalar
26     mean_iou = tf.math.reduce_mean(iou)
27
28     return mean_iou
```

The next step is to define the loss function and optimizer and finally compile the model.

```

1 # Build model.
2
3 # Get callbacks.
4 callbacks = get_callbacks(train_config)
5 # Define Loss.
6 loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False)
7 # Compile model.
8 model.compile(
9     optimizer=tf.keras.optimizers.Adam(train_config.LEARNING_RATE),
10    loss=loss_fn,
11    metrics=["accuracy", mean_iou],
12 )

```

Since we are dealing with **two** classes, we have used the `SparseCategoricalCrossentropy` loss function.

Note: The parameter `from_logits=False` has been used since the `softmax` is already implemented as the final activation layer inside the network.

Training KerasCV DeepLabv3+ on the Satellite Images of Water Bodies Dataset

We are now ready to train the model. We initiate training by calling the `model.fit()` function on the already configured data.

```

1 # Train the model, doing validation at the end of each epoch.
2 history = model.fit(
3     train_dataset,
4     epochs=train_config.EPOCHS,
5     validation_data=valid_dataset,
6     callbacks=callbacks
7 )

```

The plots show the logs for the DeepLabv3+ model with `ResNet50_v2` backbone.



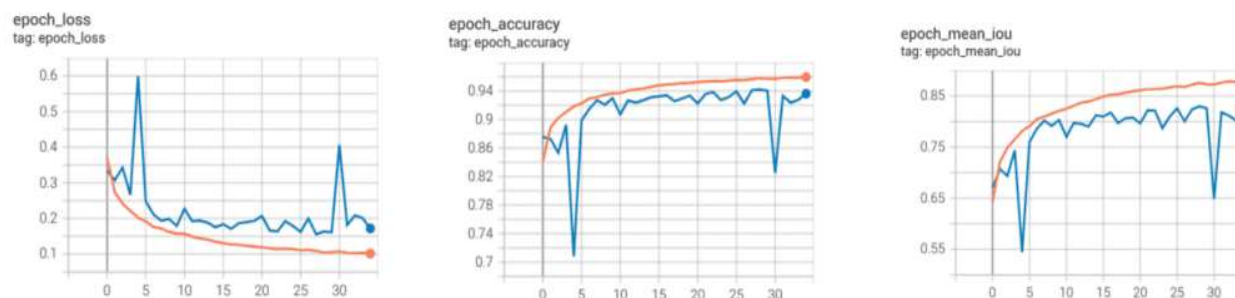
Related Post

Vision Banana: How Image Generators Are Becoming Powerful Vision Models
(<https://learnopencv.com/vision-banana-explained/>)

Vision Banana turns Nano Banana Pro into a powerful vision model for segmentation, depth estimation,

YOLO26 Keypoint Estimation: ↑
Real Time Pose Estimation

We achieved a mean pixel accuracy score of **0.9422** and a mean IoU score of **0.8299**.



(https://learnopencv.com/wp-content/uploads/2023/10/kerascv-deeplabv3-resnet50_v2-logs.png)

Training orange and Validation blue logs for DeepLabv3+ ResNet50 V2

The orange curve represents the **training** logs, while the blue ones represent the **validation** logs.

You can take a closer look at the **tensorboard logs** for the [ResNet50 V2 model](https://tensorboard.dev/experiment/iwznCDLtQuaOtX8OSmhaOQ/#scalars&_smoothingWeight=0) (https://tensorboard.dev/experiment/iwznCDLtQuaOtX8OSmhaOQ/#scalars&_smoothingWeight=0).

The plot below shows the logs for the DeepLabv3+ model with EfficientNetv2_small backbone.

We achieved a mean pixel accuracy score of **0.9399** and a mean IoU score of **0.8324**.

Real-Time Pose Estimation with Ultralytics
(<https://learnopencv.com/yolc-pose-estimation-tutorial/>)

Learn how to use YOLO26-pose with Python for real-time keypoint estimation on images and videos,

RF-DETR Segmentation: Real-Time Detection & Instance Segmentation Guide
(<https://learnopencv.com/rf-detr-segmentation-real-time-detection-instance-segmentation-guide/>)

Learn how to use RF-DETR-Seg with Python for image and video inference, understand the architecture

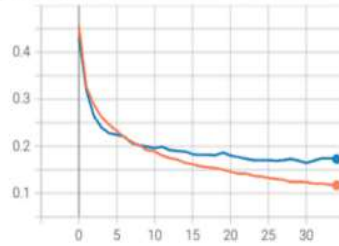
Table of Contents

- The Satellite Water Bodies Semantic Segment...
- Dataset Preparation for Semantic Segmentati...
- [Ground Truth Visualizations for Semantic Se...](#)
- Data Augmentation for KerasCV DeepLabv3+ T...
- The KerasCV DeepLabv3+ Model
- Evaluation Metrics
- Training KerasCV DeepLabv3+ on the Satellite ...
- Predictions Visualization from KerasCV DeepL...
- Summary and Conclusion
- References

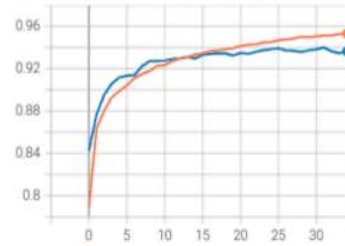
Was This Article Helpful?



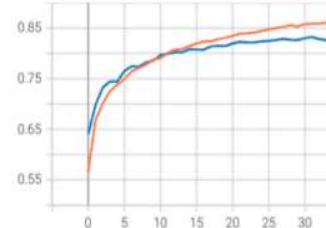
epoch_loss
tag: epoch_loss



epoch_accuracy
tag: epoch_accuracy



epoch_mean_iou
tag: epoch_mean_iou



(https://learnopencv.com/wp-content/uploads/2023/10/kerascv-deeplabv3-efficientnetv2_s-logs.png)

Training orange and Validation blue logs for DeepLabv3+ EfficientNetV2 small

This is great, considering `EfficientNetv2_small` has comparatively fewer training parameters than `ResNet50_v2`.

The **tensorboard logs** for the [EfficientNetV2_small model](https://tensorboard.dev/experiment/1kMIBedmTDOqIR7VMv4mhA/#scalars) (<https://tensorboard.dev/experiment/1kMIBedmTDOqIR7VMv4mhA/#scalars>) have also been uploaded.

From the training logs above, it can be observed that the validation plots are smoother for `EfficientNet_V2_small` compared to those of `ResNet50_V2`.

Predictions Visualization from KerasCV DeepLabv3

Now that we have performed training let us visualize a few sample predictions.

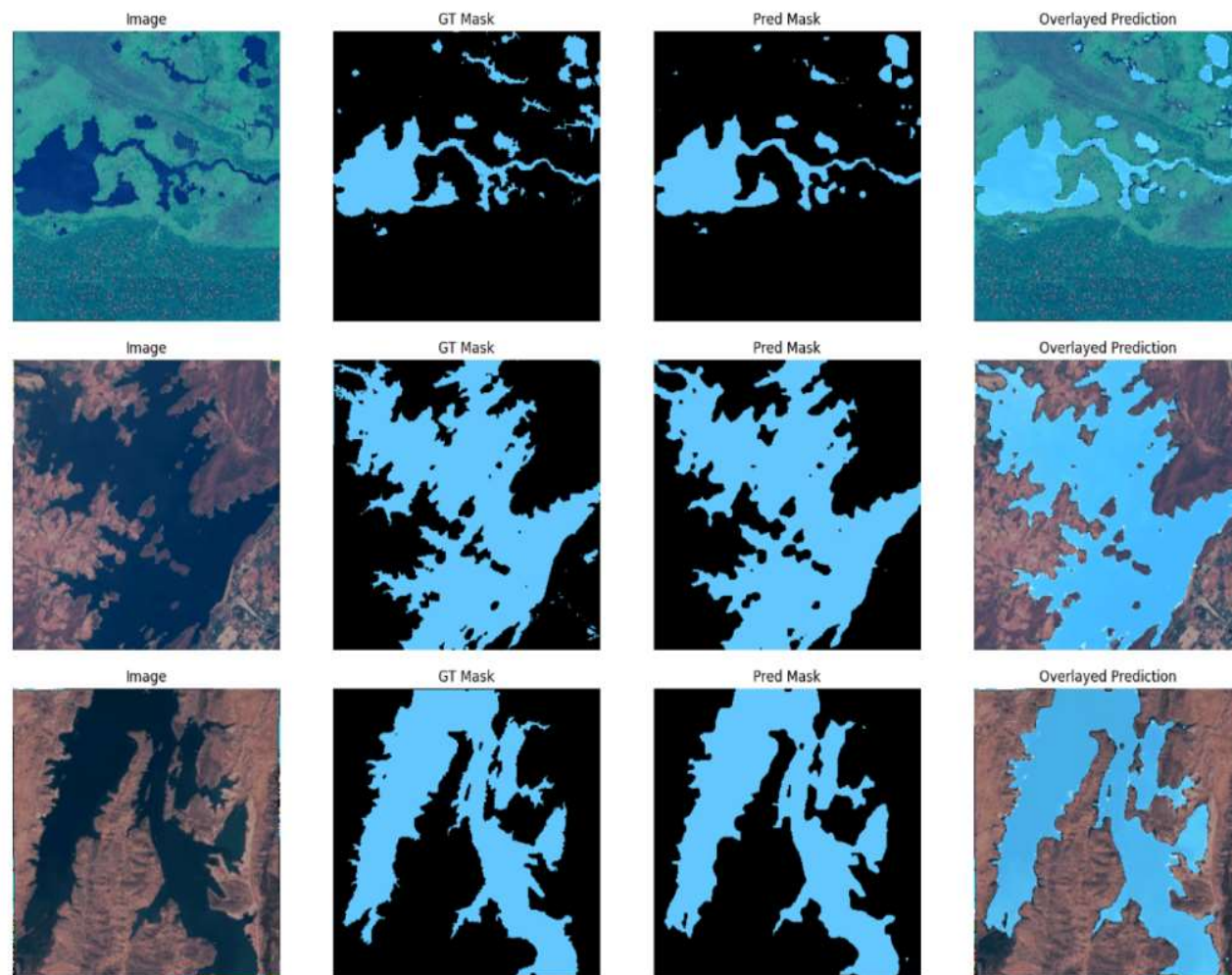
The inference plots ten prediction samples from the validation data using the fine-tuned model.

100%



```
1  def inference(model, dataset, samples_to_plot):
2
3      num_batches_to_process = 2
4      count = 0
5      stop_plot = False
6
7      titles = ["Image", "GT Mask", "Pred Mask", "Overlaid Prediction"]
8
9      for idx, data in enumerate(dataset):
10
11          if stop_plot:
12              break
13
14          batch_img, batch_mask = data[0], data[1]
15          batch_pred = (model.predict(batch_img)).astype('float32')
16          batch_pred = batch_pred.argmax(axis=-1)
17
18          batch_img = batch_img.numpy().astype('uint8')
19          batch_mask = batch_mask.numpy().squeeze(axis=-1)
20
21          for image, mask, pred in zip(batch_img, batch_mask, batch_pred):
22              count+=1
23              display_image_and_mask([image, mask, pred],
24                                     title_list=titles,
25                                     figsize=(20,8),
26                                     color_mask=True)
27
28              if count >= samples_to_plot:
29                  stop_plot=True
30                  break
```

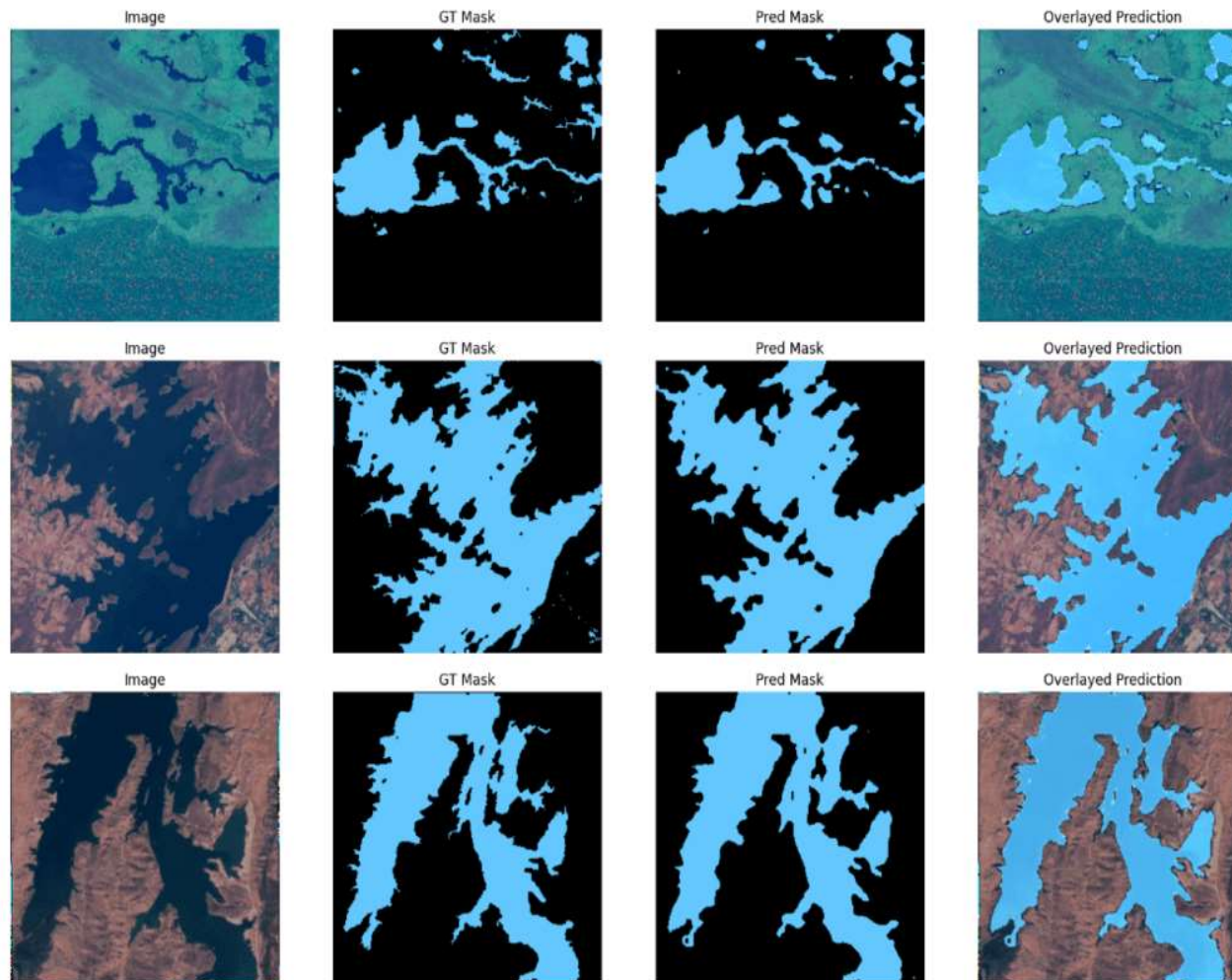
Sample predictions using the **ResNet50_v2** backbone.



(<https://learnopencv.com/wp-content/uploads/2023/10/prediction-visualization-kerascv-deeplabv3-resnet50v2.png>)

Sample Prediction for DeeLabv3+ ResNet50 V2

Sample predictions using the **EfficientNetv2_small** backbone.



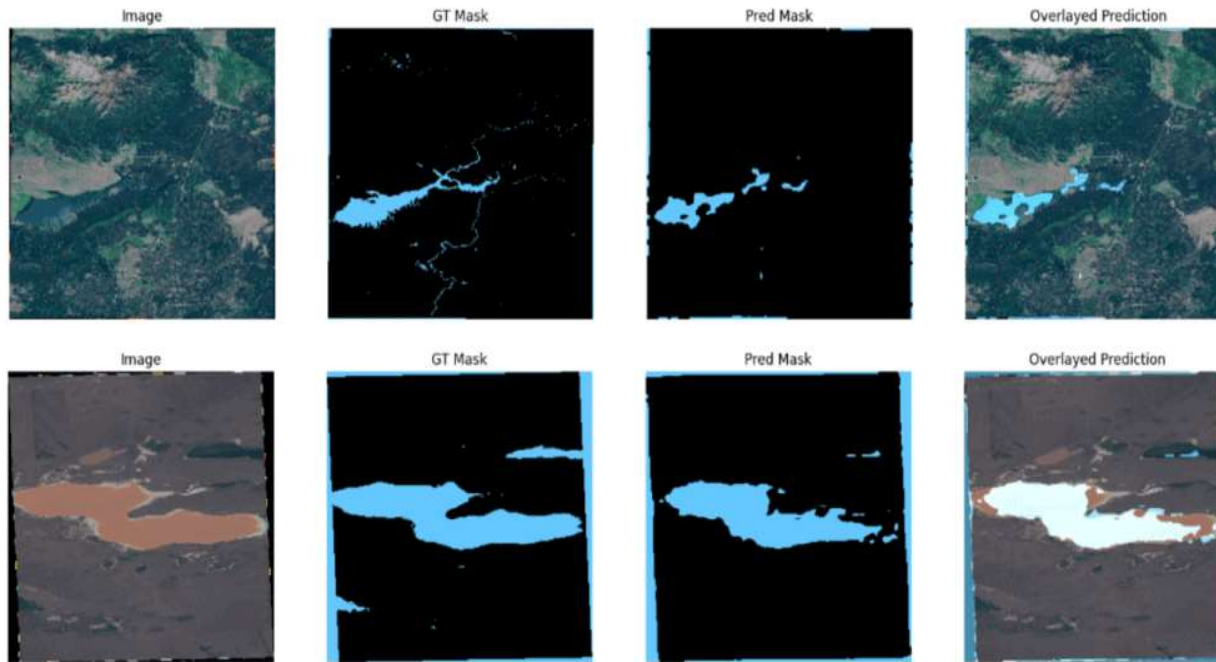
(<https://learnopencv.com/wp-content/uploads/2023/10/prediction-visualization-kerascv-deeplabv3-efficientnetv2.png>)

Sample Prediction for DeepLabv3+ EfficientNetv2 small

We can observe that although DeepLabv3+ with `EfficientNet_v2_small` has a slightly higher **mIOU** score, there aren't any significant differences in the visualizations between the two.

Both models perform really well across these samples.

Now, let's observe a couple of hard example instances from the `ResNet50v2` model.



(https://learnopencv.com/wp-content/uploads/2023/10/hard-examples-predictions-kerascv-deeplabv3-resnet50_v2.png)

Prediction Visualization across Difficult Samples for ResNet50 V2

You can see that for both images, the model predicts the boundary pixels of the input image as water bodies. However, even their corresponding ground truth masks are mislabeled; consequently, it contributed to the model misclassifying the boundary pixels.

It can be deduced that the model performs decently in these instances even though they are relatively difficult examples.

The model performance can be enhanced by addressing the data anomalies described earlier. Complex data augmentation transformations can also be added to improve the model performance.

Summary and Conclusion

To wrap up, we've covered various aspects in this article. We started by introducing DeepLabv3+ as a part of the KerasCV library. Addressing anomalies in our dataset was a key focus, and we demonstrated how to eliminate them. We prepared the dataset, after which we fine-tuned it. We also showcased how to implement different backbones on top of the DeepLabv3 segmentation head and compared the results across the different backbones. We trust that you have found this article interesting and informative. We eagerly await your thoughts and comments below!

References

- [Satellite Image of Water Bodies](https://www.kaggle.com/datasets/franciscoescobar/satellite-images-of-water-bodies) (<https://www.kaggle.com/datasets/franciscoescobar/satellite-images-of-water-bodies>)
- [KerasCV – Image Segmentation using UNet](https://keras.io/examples/vision/oxford_pets_image_segmentation/) (https://keras.io/examples/vision/oxford_pets_image_segmentation/)
- [KerasCV Repository](https://github.com/keras-team/keras-cv) (<https://github.com/keras-team/keras-cv>)
- [KerasCV Pre-trained Models and Presets](https://keras.io/api/keras_cv/models/) (https://keras.io/api/keras_cv/models/)
- [KerasCV Augmentation Layers](https://keras.io/api/keras_cv/layers/augmentation/) (https://keras.io/api/keras_cv/layers/augmentation/)

Was This Article Helpful?



Subscribe & Download Code

If you liked this article and would like to download code (C++ and Python) and example images used in this post, please [click here](#). Alternately, sign up to receive a free [Computer Vision Resource](#) Guide. In our newsletter, we share OpenCV tutorials and examples written in



 **Download Example Code**

100%



(<https://learnopencv.com/object-detection-with-deep-learning/>)
(<https://learnopencv.com/advanced-driver-assistance-systems/>)

Load Comments

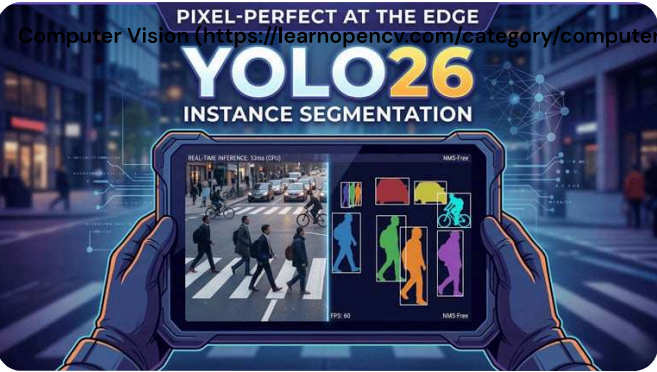


KerasCV Semantic segmentation (<https://learnopencv.com/tag/kerascv-semantic-segmentation/>)

KerasCV water Body Segmentation (<https://learnopencv.com/tag/kerascv-water-body-segmentation/>)



Read Next



(<https://learnopencv.com/yolo26-instance-segmentation-pixel-perfect-ai-at-real-time-speed/>)

[Sudip Chakrabarty](https://learnopencv.com/author/sudip/)
(<https://learnopencv.com/author/sudip/>)
on March 26, 2026

YOLO26 Instance Segmentation: Pixel-Perfect AI at Real-Time Speed
(<https://learnopencv.com/yolo26-instance-segmentation-pixel-perfect-ai-at-real-time-speed/>)

Build a complete pipeline for YOLO26 instance segmentation, from image and video inference to custom dataset training and edge deployment.

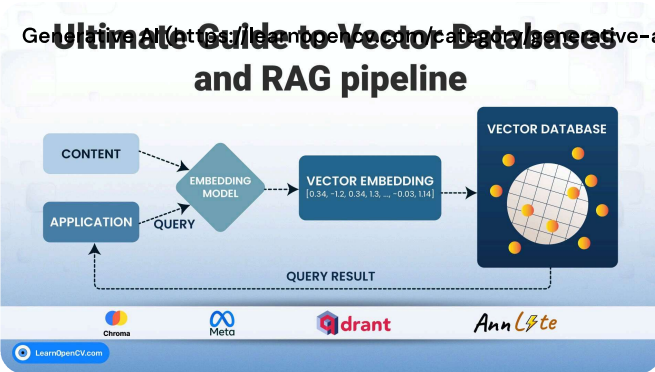


(<https://learnopencv.com/yolo26-nms-free-inference/>)

[Sudip Chakrabarty](https://learnopencv.com/author/sudip/)
(<https://learnopencv.com/author/sudip/>)
on February 10, 2026

Breaking the Bottleneck: Achieving Native NMS-Free Inference with YOLO26
(<https://learnopencv.com/yolo26-nms-free-inference/>)

YOLO26 introduces a paradigm shift with native NMS-free inference. Discover how its One-to-One label assignment eliminates post-processing overhead for stable,...



(<https://learnopencv.com/vector-db-and-rag-pipeline-for-document-rag/>)

[Kukil](https://learnopencv.com/author/kukil/) (<https://learnopencv.com/author/kukil/>)
on November 11, 2025

The Ultimate Guide to Vector DB and RAG Pipeline (<https://learnopencv.com/vector-db-and-rag-pipeline-for-document-rag/>)

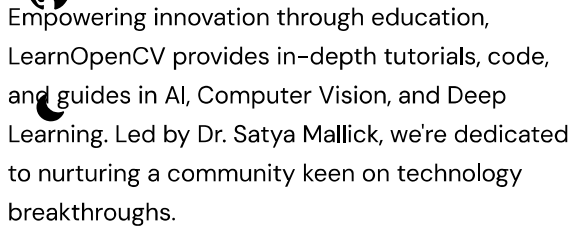
Processing long documents with VLMs or LLMs poses a fundamental challenge: input size exceeds context limits. Even with GPUs, as...

Subscribe to our Newsletter

Subscribe to our email newsletter to get the latest posts delivered right to your email.



100%



- VLM Bootcamp
- PyTorch Bootcamp
- TensorFlow & Keras Bootcamp
- OpenCV Bootcamp
- Python for Beginners

- Deep Learning
- Object Detection
- Image Classification
- YOLO
- Image Processing
- Image Segmentation

- Installation
- PyTorch
- Getting Started with OpenCV
- Keras & Tensorflow

AI Consulting
Case Studies
Get in Touch

Copyright © 2026 – BIG VISION LLC Privacy Policy (<https://learnopencv.com/privacy-policy/>) Terms and Conditions (<https://learnopencv.com/terms-and-conditions/>)







[!\[\]\(9956dc689101f4d2cd02413c5e727ccb_img.jpg\)
 \(https://www.youtube.com/c/LearnOpenCV?](https://www.youtube.com/c/LearnOpenCV?)
[!\[\]\(f293f70caa7f2c4cb2f08341cb68c804_img.jpg\)
 \(https://t.me/FaceBookOpenCV\)](https://t.me/FaceBookOpenCV)
[!\[\]\(889c96933eab7e124704cd0811cfce2f_img.jpg\)
 \(https://www.facebook.com/facebookopencv/\)](https://www.facebook.com/facebookopencv/)
[!\[\]\(a38d6bea5485490491491668e37eb4a5_img.jpg\)
 \(https://www.linkedin.com/company/facebookopencv/\)](https://www.linkedin.com/company/facebookopencv/)